TrustNode Edge — Security Architecture Note

Intended audience: customer IT / OT security reviewer

This document covers the data flow, attack surface, and controls that the TrustNode Edge agent applies on a customer site. It is intended for an IT security reviewer evaluating the agent for production deployment.

TrustNode Edge is an on-premise agent that polls industrial PLCs and power meters, stores the data locally in SQLite, and optionally mirrors it to a customer-tenant Supabase Postgres. No customer plant data is ever sent to a TrustNode-owned database; the mirror target is the customer's own cloud tenancy.

1. Network exposure

Listener	Default port + purpose
Backend HTTP	127.0.0.1:8000 — local-only by default. Tray UI and Lite/Client View talk to it over loopback.
LAN HTTP (optional)	0.0.0.0:8088-8092 — only when admin enables LAN sharing. Token + login gated. Per-user vari
OPC UA server (optional)	0.0.0.0:4840 — only when admin enables the OPC UA service. Anonymous or username/passw
MQTT broker (optional)	0.0.0.0:1883 — only when admin enables Mosquitto/amqtt. Anonymous or username/password.
Outbound to TrustNode cloud	HTTPS to trustnode.lsapps.app — control-plane only (license check, user sync). No plant data.
Outbound to customer Supabase	HTTPS (port 443) + Postgres (port 5432/6543). Customer-owned project; only their tenant_id.

2. Authentication and authorisation

Identity	Mechanism
Tray app user	Username + password against the local cp_users table. JWT issued by /api/auth/login.
LAN web viewers	Same cp_users login OR a view-link token issued by the admin per user.
Per-variant access	access_full / access_lite / access_client flags. Backend enforces via /api/lite-local/check-access
Tray menu (LAN sharing toggle)	Admin/super role only. Server-verified via /api/auth/me on every right-click — renderer cannot sp
Cloud Lite viewers	Supabase Auth (email + password). Row-Level Security restricts every read to the user's tenant_
Service-to-service	ED25519-signed license payload (private key never leaves the dev portal VPS; public key bundl

3. Data protection at rest

Data	Location + controls
SQLite buffer DB	C:\ProgramData\TrustNode\edge\trustnode_app_store.db. NTFS ACL: SYSTEM + Administrator
License + module flags	Same SQLite. License signature (ED25519) detects edits — flipping a module flag without re-sig
Generated reports	Customer-configurable directory (default Documents\TrustNode\reports). Operator can redirect t
Customer plant data on cloud	Customer's own Supabase project. Row-Level Security policies enforce tenant separation.

4. Tamper detection

- **License tamper alert** — every license-check verifies the ED25519 signature. Invalid signature writes an ERROR row to app_logs (category=license_tamper) AND surfaces a red banner in the UI.
- **Backend write veto** — historian writes are refused if the license is expired with no active trial OR the signature is invalid (Phase 3b).
- **Tray role spoofing** — the tray re-verifies the renderer's claimed user role against /api/auth/me on every menu open (Phase 3a). LAN sharing toggle requires server-confirmed admin/super.
- **Crash reporting** — if TRUSTNODE_SENTRY_DSN is set on the customer machine, uncaught Python/Node exceptions are reported to Sentry. Opt-in only; no PII sent by default.

5. Source code protection

The backend ships as a PyInstaller bundle. The output contains only .pyd (compiled extensions) and bytecode-in-archive (.pyc inside base_library.zip and the app.pkg archive). No .py source files are present on the customer install. Bytecode is stripped of docstrings and asserts (PyInstaller optimize=2).

Honest disclosure: PyInstaller bytecode can be partially recovered with publicly-available tools (uncompyle6, decompyle3). A determined attacker with administrator rights and a few hours can reconstruct large portions of the backend logic. The TrustNode license signature (ED25519 with a VPS-side private key) stops the attacker from MONETISING the recovered source — they cannot forge a license to unlock paid modules. For deployments requiring stronger code protection, a future release can compile sensitive modules via Cython AOT (no additional licensing cost). See backend/app/services/COMPILE_SENSITIVE.md.

6. Anonymous keys and public secrets

The cloud Lite (and customer-hosted Lite if used) carries the Supabase project URL and the anonymous PUBLIC key. This is by design — Supabase considers the anon key public and enforces all isolation at the database via Row-Level Security policies. The customer's auditor may flag this as 'secret in client'; the correct response is that the anon key alone CANNOT read a row outside the signed-in user's tenant.

The customer-side anon key is functionally similar to a public API endpoint URL — knowing it doesn't grant access. The SERVICE ROLE key (which bypasses RLS) is NEVER bundled in any client and lives only on the VPS in an env var.

Appendix — Disclosed limitations

Item	Status
Code signing certificate (Authenticode)	Not yet — Windows SmartScreen will show 'Unknown publisher'. Recommend $200/yr OV cert b
Crash reporting (Sentry)	Available (Phase 3d) — opt-in via TRUSTNODE_SENTRY_DSN env var.
Source obfuscation beyond PyInstaller	Documented path (Cython AOT) but not enabled by default. See COMPILE_SENSITIVE.md.
Status page	Not yet.
Automated security scanning of dependencies	Dependabot